Environment Setup

Mongosh

use aggregation_lab

db.orders.insertMany([
{ _id: 1, productId: 101, quantity: 2, price: 25.00, customerId: "A123", orderDate: ISODate("2024-01-15T00:00:00Z") },
{ _id: 2, productId: 102, quantity: 1, price: 50.00, customerId: "B456", orderDate: ISODate("2024-01-20T00:00:00Z") },
{ _id: 3, productId: 101, quantity: 3, price: 25.00, customerId: "A123", orderDate: ISODate("2024-01-25T00:00:00Z") },
{ _id: 4, productId: 103, quantity: 1, price: 75.00, customerId: "C789", orderDate: ISODate("2024-02-01T00:00:00Z") },
{ _id: 5, productId: 102, quantity: 2, price: 50.00, customerId: "B456", orderDate: ISODate("2024-02-10T00:00:00Z") }
])

db.products.insertMany([
{ _id: 101, name: "Widget A", category: "Widgets" },
{ _id: 102, name: "Widget B", category: "Widgets" },
{ _id: 103, name: "Gadget X", category: "Gadgets" }
])

Step-by-Step Instructions

Step 1: Basic $group Stage: Calculate Total Order Count

Description:
This step counts how many order records exist in the collection.

Command: db.orders.aggregate([{ $group: { _id: null, totalOrders: { $sum: 1 } } }])

```
aggregation_lab> db.orders.aggregate([ { $group: { _id: null, totalOrders: { $sum: 1 } } } ])
[ { _id: null, totalOrders: 5 } ]
```

Explanation:
All documents are grouped into a single bucket, and each document contributes a value of 1. The final result represents the total number of orders stored in the database.

Step 2: $group with $sum and Grouping by productId

Description:
This step calculates how many items were ordered for each product.

Command: db.orders.aggregate([{ $group: { _id: "$productId", totalQuantity: { $sum: "$quantity" } } }])

```
aggregation_lab> db.orders.aggregate([ { $group: { _id: "$productId", totalQuantity: { $sum: "$quantity" } } } ])
[
  { _id: 102, totalQuantity: 3 },
  { _id: 101, totalQuantity: 5 },
  { _id: 103, totalQuantity: 1 }
]
```

Explanation:
Documents are grouped by product identifier. For each group, the quantities are added together to show how many units of each product have been sold.

Step 3: $group with Multiple Accumulators: Total Quantity and Average Price

Description:
This step summarizes both total quantity and average price per product.

Command: db.orders.aggregate([{ $group: { _id: "$productId", totalQuantity: { $sum: "$quantity" }, averagePrice: { $avg: "$price" } } }])

```
aggregation_lab> db.orders.aggregate([
| {
| $group: {
| _id: "$productId",
| totalQuantity: { $sum: "$quantity" },
| averagePrice: { $avg: "$price" }
| }
| }
| ])
[
  { _id: 102, totalQuantity: 3, averagePrice: 50 },
  { _id: 101, totalQuantity: 5, averagePrice: 25 },
  { _id: 103, totalQuantity: 1, averagePrice: 75 }
]
```

Explanation:
For every product group, the system calculates two things: the total number of items sold and the average price across all related orders. This helps understand both sales volume and pricing trends.

Step 4: Basic $lookup Stage: Joining orders and products Collections

Description:
This step combines order data with product details.

Command: db.orders.aggregate([{ $lookup: { from: "products", localField: "productId", foreignField: "_id", as: "productInfo" } }])

```
aggregation_lab> db.orders.aggregate([ { $lookup: { from: "products", localField: "productId", foreignField: "_id", as: "productInfo" } } ])
[
  {
    _id: 1,
    productId: 101,
    quantity: 2,
    price: 25,
    customerId: 'A123',
    orderDate: ISODate('2024-01-15T00:00:00.000Z'),
    productInfo: [ { _id: 101, name: 'Widget A', category: 'Widgets' } ]
  },
  {
    _id: 2,
    productId: 102,
    quantity: 1,
    price: 50,
    customerId: 'B456',
    orderDate: ISODate('2024-01-20T00:00:00.000Z'),
    productInfo: [ { _id: 102, name: 'Widget B', category: 'Widgets' } ]
  },
  {
    _id: 3,
    productId: 101,
    quantity: 3,
    price: 25,
    customerId: 'A123',
    orderDate: ISODate('2024-01-25T00:00:00.000Z'),
    productInfo: [ { _id: 101, name: 'Widget A', category: 'Widgets' } ]
  },
  {
    _id: 4,
    productId: 103,
    quantity: 1,
    price: 75,
    customerId: 'C789',
    orderDate: ISODate('2024-02-01T00:00:00.000Z'),
    productInfo: [ { _id: 103, name: 'Gadget X', category: 'Gadgets' } ]
  },
  {
    _id: 5,
    productId: 102,
    quantity: 2,
    price: 50,
    customerId: 'B456',
    orderDate: ISODate('2024-02-10T00:00:00.000Z'),
    productInfo: [ { _id: 102, name: 'Widget B', category: 'Widgets' } ]
  }
]
```

Explanation:

Each order is matched with its corresponding product using the product ID. The result adds extra information (like name and category) into each order record.

Step 5: Combining $lookup and $unwind Stages

Description:

This step simplifies the joined data structure.

Command: db.orders.aggregate([{ $lookup: { from: "products", localField: "productId", foreignField: "_id", as: "productInfo" } }, { $unwind: "$productInfo" }])

```
aggregation_lab> db.orders.aggregate([ { $lookup: { from: "products", localField: "productId", foreignField: "_id", as: "productInfo" } }, { $unwind: "$productInfo" } ])
[
  {
    _id: 1,
    productId: 101,
    quantity: 2,
    price: 25,
    customerId: 'A123',
    orderDate: ISODate('2024-01-15T00:00:00.000Z'),
    productInfo: { _id: 101, name: 'Widget A', category: 'Widgets' }
  },
  {
    _id: 2,
    productId: 102,
    quantity: 1,
    price: 50,
    customerId: 'B456',
    orderDate: ISODate('2024-01-20T00:00:00.000Z'),
    productInfo: { _id: 102, name: 'Widget B', category: 'Widgets' }
  },
  {
    _id: 3,
    productId: 101,
    quantity: 3,
    price: 25,
    customerId: 'A123',
    orderDate: ISODate('2024-01-25T00:00:00.000Z'),
    productInfo: { _id: 101, name: 'Widget A', category: 'Widgets' }
  },
  {
    _id: 4,
    productId: 103,
    quantity: 1,
    price: 75,
    customerId: 'C789',
    orderDate: ISODate('2024-02-01T00:00:00.000Z'),
    productInfo: { _id: 103, name: 'Gadget X', category: 'Gadgets' }
  },
  {
    _id: 5,
    productId: 102,
    quantity: 2,
    price: 50,
    customerId: 'B456',
    orderDate: ISODate('2024-02-10T00:00:00.000Z'),
    productInfo: { _id: 102, name: 'Widget B', category: 'Widgets' }
  }
]
```

Explanation:
Since the join result is stored as an array, this stage converts it into a single embedded object. This makes the data easier to read and process in the next steps.

Step 6: Combining $lookup and $group to Calculate Total Sales by Category

Description:
This step calculates total sales based on product categories.

Command: db.orders.aggregate([{ $lookup: { from: "products", localField: "productId", foreignField: "_id", as: "productInfo" } }, { $unwind: "$productInfo" }, { $group: { _id: "$productInfo.category", totalSales: { $sum: { $multiply: ["$quantity", "$price"] } } } }])

```
aggregation_lab> db.orders.aggregate([ { $lookup: { from: "products", localField: "productId", foreignField: "_id", as: "productInfo" } }, { $unwind: "$productInfo" }, { $group: { _id: "$p
roductInfo.category", totalSales: { $sum: { $multiply: ["$quantity", "$price"] } } } } ])
[
  { _id: 'Widgets', totalSales: 275 },
  { _id: 'Gadgets', totalSales: 75 }
]
```

Explanation:
After combining order and product data, records are grouped by category. For each category, total revenue is calculated by multiplying quantity and price, then summing the results. This provides insight into which category generates the most sales.

Challenges & Solutions

Challenges

- Data not appearing in $lookup
- $group results are inconsistent
- Error due to data type

Solutions

- Reject localField and ForeignField values with the same data type
- Check collection names are correct
- Use $unwind to simplify data join processing
- Differentiate between numeric fields used in $sum and $avg